TCP Congestion Control

When the load offered to any network is more than it can handle, congestion builds up. The Internet is no exception. The network layer detects congestion when queues grow large at routers and tries to manage it, if only by dropping packets. It is up to the transport layer to receive congestion feedback from the network layer and slow down the rate of traffic that it is sending into the network.

AIMD (Additive Increase Multiplicative Decrease) congestion control principle,

A transport protocol that uses the AIMD (Additive Increase Multiplicative Decrease) algorithm to respond to congestion signals from the network can gradually achieve fair and efficient bandwidth distribution among multiple users. TCP maintains a congestion window whose size is the number of bytes the sender may have in the network at any time. The corresponding rate is the window size divided by the round-trip time of the connection. TCP adjusts the size of the window according to the AIMD rule.

$$\text{Sending Rate} = \frac{\text{Congestion Window}}{\text{RTT}}$$

TCP maintains two separate windows:

Congestion Window (cwnd) and Flow Control Window (rwnd). Flow control window which specifies the number of bytes that the receiver can buffer. Both windows are tracked in parallel, and the number of bytes that may be sent is the smaller of the two windows.

The actual amount of data that can be sent is:

$$\boxed{\text{Effective Window} = \min(cwnd, rwnd)}$$

If the receiver says ‘‘send 64 KB’’ but the sender knows that bursts of more than 32 KB clog the network, it will send 32 KB. On the other hand, if the receiver says ‘‘send 64 KB’’ and the sender knows that bursts of up to 128 KB get through effortlessly, it will send the full 64 KB requested.

Example 1:

Congestion window = 32 KB
Flow control window = 64 KB
Effective window = 32 KB.

Example 2:

Congestion window = 128 KB
Flow control window = 64 KB
Effective window = 64 KB.

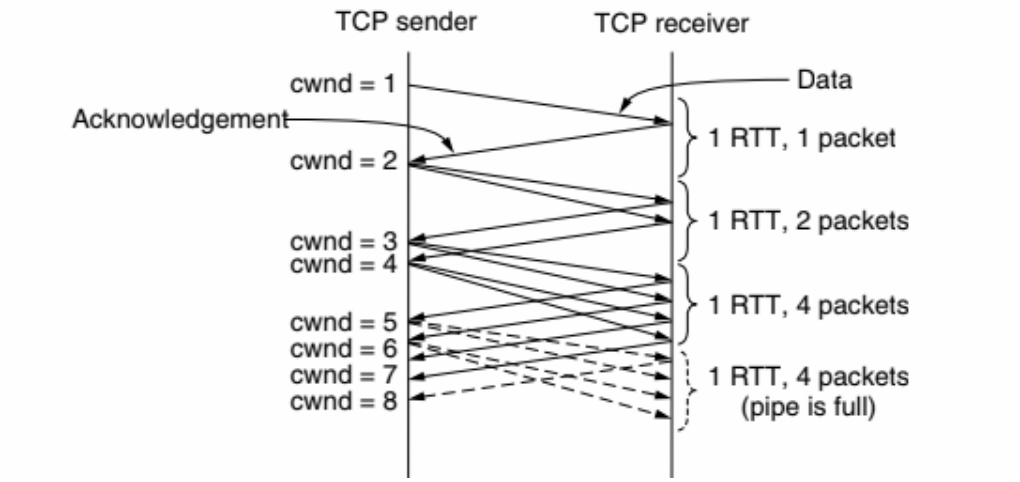
Therefore, TCP always uses the smaller window.

TCP Congestion control works in 3 stages:

Slow Start:

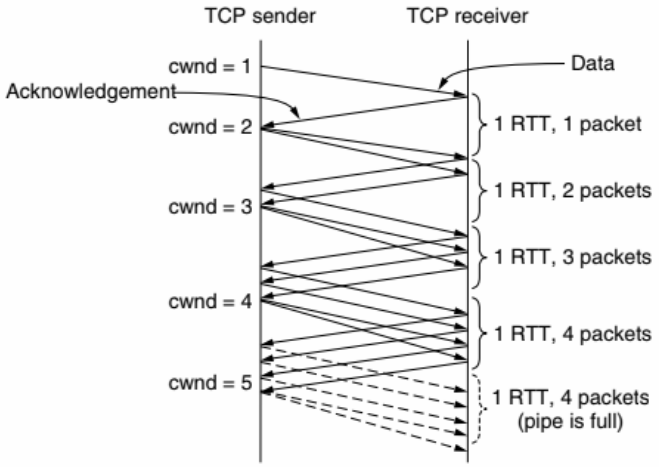
When a connection is established, the sender initializes the congestion window to a small initial value of at most four segments; The packets will take a round-trip time to be acknowledged. For each segment that is acknowledged before the retransmission timer goes off, the sender adds one segment’s worth

of bytes to the congestion window. The upshot is that every acknowledged segment allows two more segments to be sent. The congestion window is doubling every roundtrip time. This algorithm is called slow start, To keep slow start under control, the sender keeps a threshold for the connection called the slow start threshold. Initially this value is set arbitrarily high, to the size of the flow control window, Whenever a packet loss is detected, for example, by a timeout, the slow start threshold is set to be half of the congestion window and the entire process is restarted.



2. **Additive Increase (AI)**

Whenever the slow start threshold is crossed, TCP switches from slow start to additive increase. In this mode, the congestion window is increased by one segment every round-trip time.



3. **Multiplicative Decrease (MD):**

TCP monitors packet delivery and acknowledgement (ACK) behaviour to detect network congestion. When packets are delivered successfully, TCP gradually increases the congestion window (cwnd) to improve the transmission rate. However, when packet loss is detected, TCP assumes that network congestion has occurred and reduces the congestion window to decrease the sending rate and prevent further congestion.

TCP detects packet loss using mechanisms such as: Retransmission timeout (RTO) and Duplicate acknowledgements.

1. **Retransmission Timeout(RTO) :** When the sender transmits a packet, it expects an acknowledgement (ACK) from the receiver.If the ACK does not arrive within the retransmission timeout period, TCP assumes that the packet may have been lost due to congestion.TCP retransmits the lost packet.It reduces the congestion window to control network traffic.

2. **Duplicate Acknowledgements** When a receiver gets packets out of order, it sends acknowledgements with the same acknowledgement number repeatedly. These are called **duplicate acknowledgements (Duplicate ACKs)**. TCP uses three duplicate acknowledgements as an indication that a packet has probably been lost. When a packet is lost, the receiver continues receiving later packets. Since a packet is missing, the receiver repeatedly sends ACKs with the same acknowledgement number. When the sender receives three duplicate ACKs, it assumes that a packet has been lost

TCP Tahoe

It is a version of TCP congestion control introduced in the 4.2BSD Tahoe release in 1988. TCP Tahoe is a TCP congestion control algorithm that uses AIMD principles along with Slow Start and Fast Retransmission.TCP Tahoe responds to packet loss by reducing the congestion window to one MSS, regardless of whether the loss is detected through a timeout or three duplicate ACKs.

TCP Tahoe uses three main mechanisms:

1. Slow Start
2. Congestion Avoidance
3. Fast Retransmission

1. **Slow Start:** Initially, the congestion window (cwnd) is set to 1 MSS. It increases exponentially approximately doubles every RTT) until it reaches the slow start threshold (ssthresh).

2. **Congestion Avoidance:** When the congestion window reaches the threshold, TCP increases the window linearly (additive increase) to avoid congestion.

3. **Fast Retransmission:** When the sender receives three duplicate acknowledgements (ACKs), it assumes that a packet is lost and retransmits the missing packet immediately without waiting for a timeout.

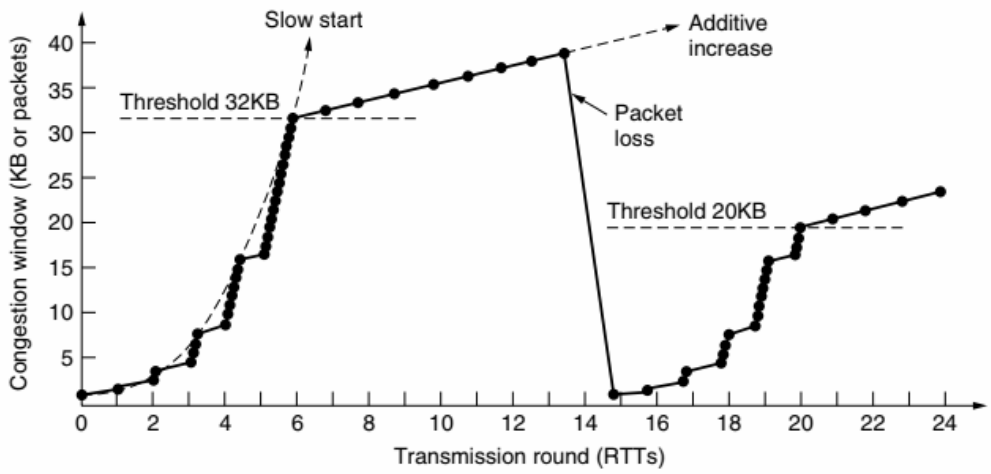


Figure 6-46. Slow start followed by additive increase in TCP Tahoe.

When congestion occurs, TCP Tahoe handles it in the following way:

When packet loss is detected through either a **timeout or three duplicate ACKs**, the following actions are performed:

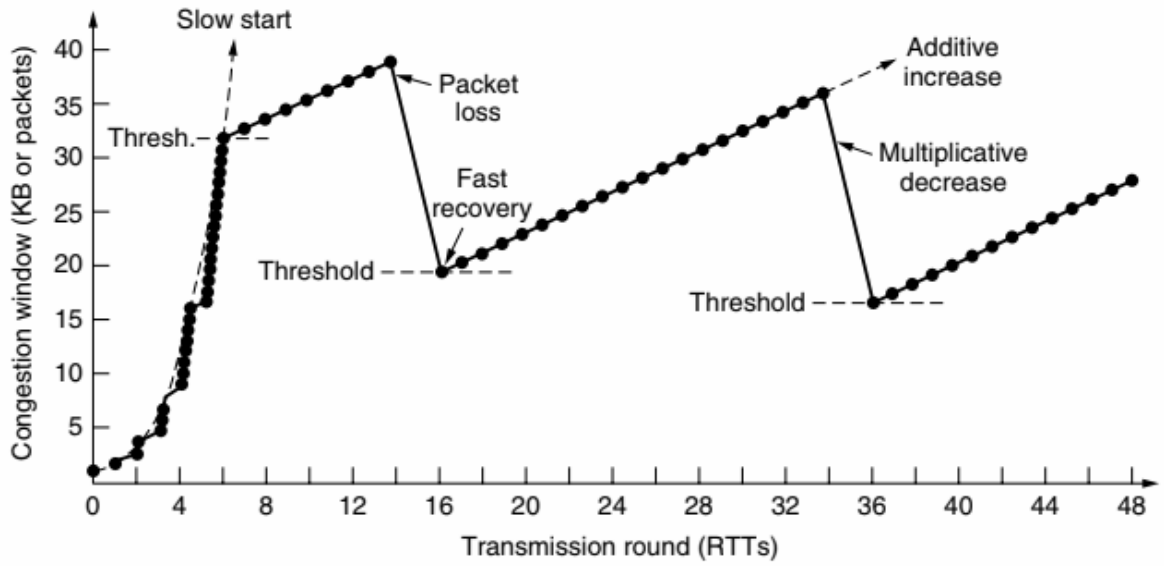
- The slow start threshold (ssthresh) is set to half of the current congestion window (cwnd).
- The congestion window (cwnd) is reset to 1 MSS.
- TCP restarts the Slow Start phase.

TCP Reno

TCP Reno is an improved version of TCP Tahoe introduced to improve network performance and handle congestion more efficiently. TCP Reno release in 1990. In TCP Tahoe, whenever packet loss is detected through three duplicate ACKs, the congestion window is reset to 1 MSS, and Slow Start begins again. This significantly reduces the sending rate, even when the network is not severely congested.To solve this problem, TCP Reno introduced a new mechanism called **Fast Recovery**.

It uses four main mechanisms:

1. Slow Start,
2. Congestion Avoidance,
3. Fast Retransmission
4. Fast Recovery.



1. Slow Start

Initially, the congestion window (cwnd) is set to 1 MSS.
The congestion window increases exponentially (approximately doubles every RTT).
This continues until the slow start threshold (ssthresh) is reached.

2. Congestion Avoidance

When cwnd reaches ssthresh, TCP switches to Congestion Avoidance.
The congestion window increases linearly (additive increase).
This helps prevent network congestion.

3.Fast Retransmission

When the sender receives three duplicate ACKs, it assumes that a packet is lost.
It immediately retransmits the missing packet without waiting for a timeout.

4.Fast Recovery

It is a congestion control mechanism used in TCP Reno to recover from packet loss without restarting the Slow Start phase.When the sender receives **three duplicate acknowledgements (ACKs)**, it

assumes that a packet is lost. Instead of reducing the congestion window to 1 MSS, TCP Reno reduces it approximately by half and continues transmitting data.

.